

Real-PR Run Comparison — Old Single-Pass vs. New Chunked + Dependency Waves

Subject PR: [Ohanafy/OHFY-Split #560](#) — feat(ecom): Phase 2 — pricing/promo completion & checkout (BMS-5576)
Change under test: [#578](#) — Phase 0.5 scaling + dependency-ordered wave chunking of `/code-review`
Posted review (new run): [pullrequestreview-4725420335](#) — 7 inline + full body, COMMENT only

This is the “measure” step from Lesson 2 — the one that turns “I think this helps” into observed data. It is a **single** before/after run, not a statistical result. Treat it as one honest data point.

The PR under review

12,228 changed lines across 100 files — a genuine four-tier dependency chain plus shared paths:

PACKAGE	FILES	~CHANGED LINES	TIER / DEPS
OHFY-Data-Model	20	481	0
OHFY-OMS	4	805	3
OHFY-eCommerce	14	1,478	3 — deps: Data-Model, OMS
OHFY-eCommerce-UI	38	6,547	4 — deps: all above
shared (docs / orgScripts / org-metadata / .claude)	~31	2,303	catch-all

Derived wave plan (from `sfdx-project.json`, no hardcoded names): **W1** Data-Model + shared → **W2** OMS → **W3** eCommerce → **W4** eCommerce-UI.

Method (kept as fair as possible)

- Old / “before”:** 3 global finder agents (correctness; cross-file + contracts; cleanup + altitude), each reading the **whole** ~14k-line diff, capped ~15 findings after dedup. All parallel (one wave). Sonnet-pinned.
- New / “after”:** 5 chunk agents (one per package + catch-all), each running the full angle set + verify on **only its chunk**, executed in the 4 dependency waves above, each downstream wave receiving its touched ancestors’ **contract-change digest**. Sonnet-pinned. Consolidated in the main loop.

- Both reviewed the same range: base `2954f6af6` → PR head `792b6d664` .

Cost & coverage

	OLD (SINGLE-PASS)	NEW (CHUNKED WAVES)
Agents	3 (all parallel)	5 (across 4 sequential waves)
Tokens	~418k	~480k
Wall-clock	~1 wave	~4 waves (sequencing tax)
Coverage (self-reported)	every finder skimmed the non-Apex ~40% (metadata, docs, Playwright)	full pass per package except the 6,547-line eCommerce-UI chunk, which still skimmed

Findings

Both runs caught (the key result)

- BLOCKER** PDP reprice vs. tier-meter price disagreement — `ecomProductPage.js` reprices with only the current item's qty while the meter pools cart-wide in-scope qty → shopper sees an “unlocked” price the resolver won't charge. Both flagged BLOCKER.
- Doubled pricing waterfall** — `DraftInvoiceController` runs the full `S_PriceResolver` resolve twice per confirm. Old: BLOCKER; New: **WARNING** (perf, not correctness — legitimate disagreement).

Only the NEW run found (breadth / cross-tier)

- Cross-tier tax shift** — `Tax_Authority__c` sharing Private→Read (Data-Model) rode the digest into the OMS chunk, which flagged that guest checkout tax totals change on merge with no test coverage. Old run never connected them. **The digest's clearest win.**
- 3 shared / deploy findings** — guest sharing-rule name mismatch (`E_Commerce` vs `E_Commerce1`) that silently breaks banners for logged-out users; stale `LEARNINGS.md` ; fragile `unpackaged/unpackaged` retrieve path. The catch-all got a dedicated agent instead of being skimmed last.
- SYSTEM_MODE rule-6 misuse** in `StorefrontBannerController` — old confirmed a citation existed; new caught it was misapplied.

Only the OLD run found (chunking's real weakness)

- **Cross-package duplication.** The whole-tree pass caught **three** copy-paste instances across packages (`scopeMatches` , banner-group query, fieldset-resolution). The chunked run saw only within-chunk files, so it caught the one visible in a single chunk and **missed the cross-package ones**. The “chunking loses cross-chunk findings” caveat, observed live.

Verdict — iterate, do not rebuild

CONCLUSION

The foundation is sound. On headline high-severity bugs the two runs **tied**; the new run's edge is **breadth for a first sweep** (dedicated low-glamour coverage + the cross-tier tax signal), at the cost of **more tokens and much more wall-clock**, with **one genuine regression vs. old: cross-package duplication detection**.

Two small extensions (not a teardown), mapped to the Lesson-2 knobs:

- **Size cap → sub-chunk oversized packages.** The 6,547-line eCommerce-UI chunk reproduced the original context-saturation problem in miniature. Package chunking bounds *count*, not *size*. (Chunk-boundary knob.)
- **Add a cheap whole-tree duplication pass** in consolidation (even mechanical/grep-based), since chunking is structurally blind to cross-package copy-paste. (Consolidation step.)

One thing to **watch, not fix yet**: the wave/digest machinery is the most complex and most expensive part and produced ~one clear unique win here (the tax finding). If further runs show the digest rarely catches something cheaper wouldn't, collapse waves → **flat parallel chunks + a shared mechanical contract index** (changed public signatures/fields extracted up front, handed to all chunks): same cross-tier awareness, no sequencing tax. Keep waves for now; re-evaluate after a few more real PRs.

Caveats on this record

- One PR, one run each — not a statistical comparison. Severity calls and even finding sets vary run to run.
- The “old” run got a fair shot (3 focused finders, allowed to self-parallelize), so it is not a strawman — which is *why* the results are close.
- No findings were verified against ground truth (post-merge fixes/reverts). A stronger future measurement: score catch-rate against a merged PR whose real bugs are known in hindsight.

Companion to `run-comparison-pr560.md` and `lesson-2-scaling-large-prs.html` in the same workspace. Generated 2026-07-17.